

NIS4307 人工智能导论 (A 类)

作业报告

任务名称 基于 BERTweet、RAG 与大语言模型的
可解释谣言检测系统

完成组号 4

小组人员 陈忠淳 沈润泽 戴铭辰 谢紫浩

完成时间 2026 年 6 月 4 日——2026 年 6 月 14 日

目录

1 任务目标	3
2 系统设计与实现	3
2.1 实施方案	3
2.2 核心代码分析	3
2.2.1 数据处理模块——src/data.py	4
2.2.2 模型定义与推理——src/model.py、src/predict.py	4
2.2.3 训练流程——src/train.py	4
2.2.4 RAG 检索增强——src/rag_service.py	5
2.2.5 LLM 多阶段分析——src/api_service.py	5
2.2.6 FastAPI 前端集成——src/app.py	5
2.3 检测结果分析	7
2.4 判断依据的分析	8
3 工作总结	8
3.1 收获与心得	8
3.2 问题与解决	9
3.3 分工与合作	9
4 课程建议	9

1 任务目标

本项目围绕“可解释的谣言检测”任务展开，目标是构建一个能够对英文推文文本进行自动检测并给出判断依据的系统。

系统输入为一段英文文本；输出包括两部分：一是二分类检测结果，其中 0 表示非谣言，1 表示谣言；二是自然语言形式的判断依据，用于解释系统作出该判断的主要原因。

2 系统设计与实现

2.1 实施方案

本项目采用 **BERTweet 小模型分类 + RAG 检索增强 + 大语言模型分析 + 前端展示** 的复合架构。其流程如图 1 所示：

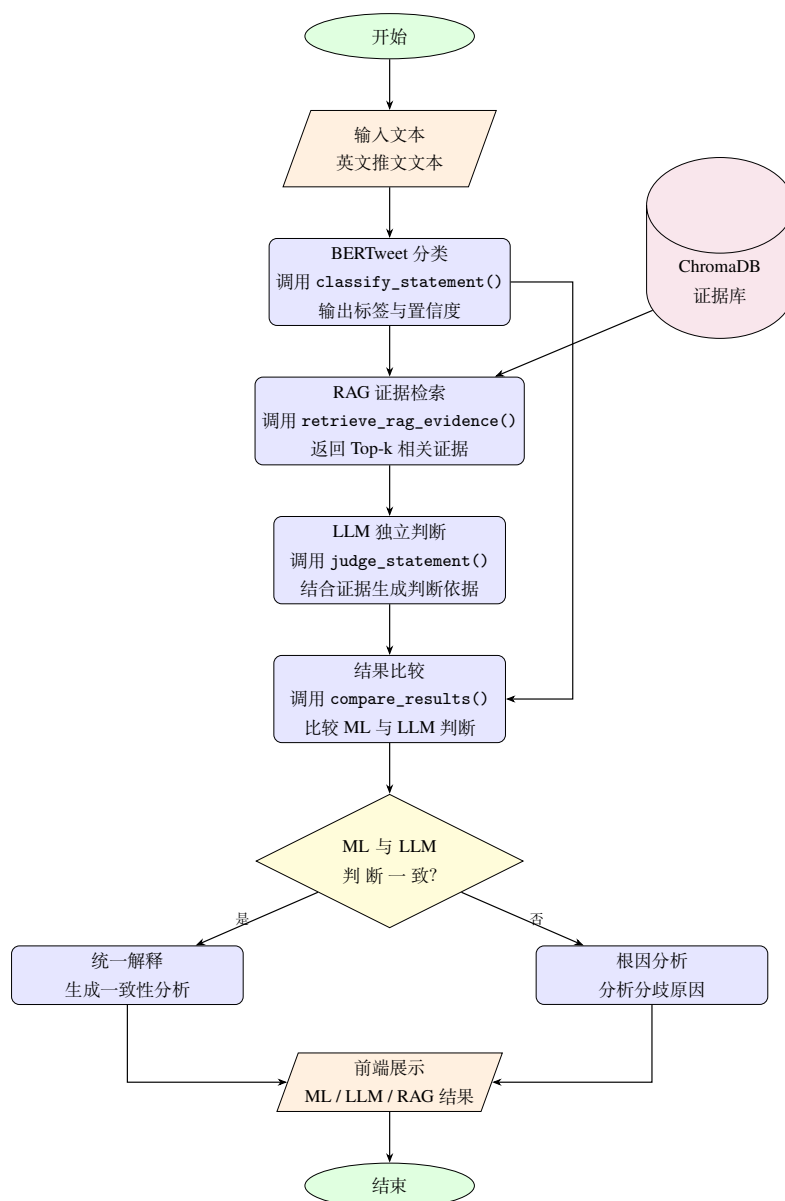


图 1: 系统处理流程图

系统最终通过前端页面展示完整检测结果，包含以下 5 部分：

- 机器学习模型的输出判断及其置信度；
- 大语言模型对于输入文本的独立输出判断；
- 大语言模型对于前两者分析结果的比较及分析；
- 检索增强生成所得到的相关证据摘要。

2.2 核心代码分析

本项目代码位于 `src/` 目录下，按功能划分为数据处理、模型定义与推理、训练流程、RAG 检索、LLM 多阶段分析和前端集成六个模块。以下逐一分析各模块的核心实现。

2.2.1 数据处理模块——`src/data.py`

数据处理模块负责读取 `train.csv` 和 `val.csv`，并完成文本清洗、标签校验、重复样本和冲突样本处理。`load_datasets()` 调用 `_clean_split()` 分别清洗训练集与验证集，并额外删除训练集与验证集之间的重叠文本，降低数据泄漏风险。`TweetDataset` 负责将清洗后的文本封装为 PyTorch 数据集，在 `__getitem__()` 中调用 `tokenizer` 生成 `input_ids`、`attention_mask` 和 `labels`，供模型训练与验证使用。

2.2.2 模型定义与推理——`src/model.py`、`src/predict.py`

模型基于 Hugging Face 的 `AutoModelForSequenceClassification` 实现，分类头输出 2 类概率。`model.py` 采用懒加载方式，在首次调用 `classify_statement()` 时加载 `checkpoints/bertweet/best_model/` 中的最佳模型和分词器，并切换到 `eval()` 模式。推理时系统先对输入文本进行规范化和编码，再在 `torch.no_grad()` 下前向传播，对 logits 做 softmax，返回预测标签和置信度。`predict.py` 提供命令行预测接口，便于离线测试。

2.2.3 训练流程——`src/train.py`

训练脚本完成模型微调、验证评估和最佳模型保存。优化器使用 AdamW，学习率为 $1e-5$ ，并配合线性预热衰减调度。训练过程中每个 epoch 后在验证集上计算 accuracy 和 macro-F1，并以 macro-F1 作为 early stopping 指标；当指标连续若干轮未提升时停止训练，保存最佳模型、tokenizer 和训练指标。`plot_history.py` 用于生成训练曲线，辅助分析过拟合情况。

2.2.4 RAG 检索增强——`src/rag_service.py`

RAG 模块用于为 LLM 解释提供参考证据。`retrieve_rag_evidence()` 动态调用 `RAG/chroma_retriever.py` 中的 `ChromaRetriever`，从本地 ChromaDB 中检索 Top-k 相似证据，并返回来源、标签、距离和文本内容。RAG 不直接参与 BERTweet 分类；若数据库缺失或依赖异常，函数返回空列表，保证主检测流程仍可运行。

2.2.5 LLM 多阶段分析——`src/api_service.py`

LLM 分析模块是整个系统可解释性的核心，实现了三阶段提示工程管道：

LLM 模块采用三阶段分析。第一阶段 `judge_statement()` 结合输入文本和 RAG 证据进行独立判断，并输出结构化 JSON；第二阶段 `compare_results()` 比较 ML 与 LLM 结果是否一致；第三阶段 `analyze_root_cause()` 根据一致或分歧情况生成统一解释或根因分析。为了提高稳定性，`_parse_json()` 对 LLM 返回结果进行容错解析，尽量从普通 JSON、Markdown 代码块或文本片段中提取结构化字段。

2.2.6 FastAPI 前端集成——src/app.py

app.py 使用 FastAPI 组织完整流程。/analyze 端点依次调用 BERTweet 分类、RAG 检索、LLM 独立判断、结果比较和根因分析，并将 ML Verdict、LLM Verdict、Confidence、Reasoning、Comparison 和 RAG Evidence 传入 Jinja2 模板。前端页面最终展示检测结果、解释文本和检索证据，如图2所示。

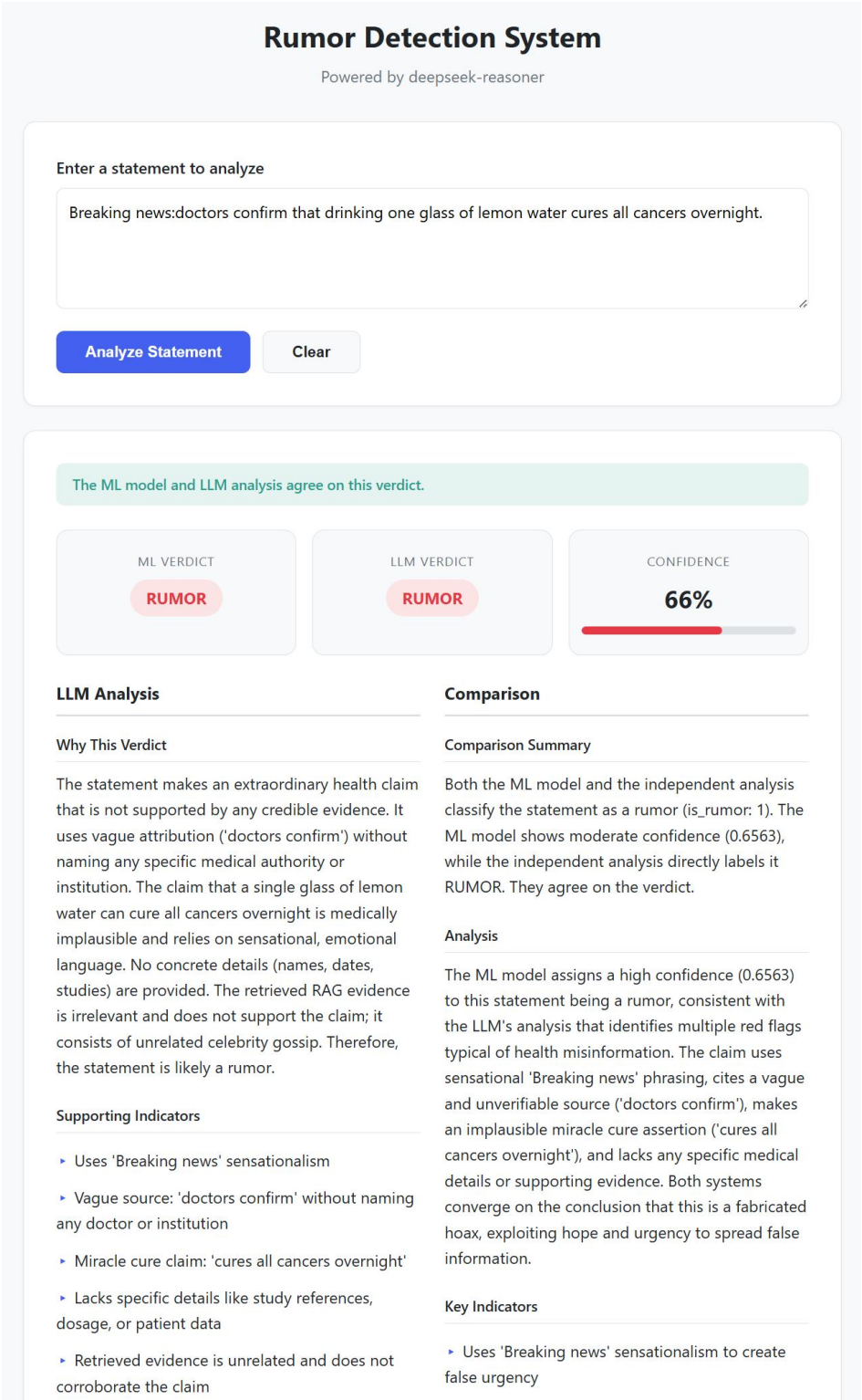


图 2: 前端页面展示示例

2.3 检测结果分析

本项目使用 `train.csv` 微调 BERTweet 模型，并在 `val.csv` 上验证。数据清洗后，训练集共 2832 条样本（非谣言 1596 条，谣言 1236 条），验证集共 400 条样本（非谣言 225 条，谣言 175 条），类别分布基本均衡，无明显长尾偏差。训练配置为学习率 1×10^{-5} 、batch size 32、最大训练 16 个 epoch，并以验证集 macro-F1 作为 early stopping 指标，patience=2，用连续两个 epoch 未提升触发停止。

实验结果显示，模型在第 5 个 epoch 取得最佳验证集表现：accuracy 为 0.8700，macro-F1 为 0.8688，验证集 loss 为 0.3696。早停机制在第 7 个 epoch 后触发，系统最终保存第 5 轮 macro-F1 最优的模型权重作为最终检测模型。

图 3、图 4 与图 5 分别展示验证集 accuracy、macro-F1 以及训练/验证 loss 随 epoch 的变化。从曲线可见，accuracy 与 macro-F1 在前 5 轮持续上升，第 5 轮达到峰值后回落并趋于稳定；训练 loss 从 0.6735 下降到 0.1256，说明模型持续拟合训练集，而验证 loss 在第 5 轮降至 0.3696 后开始反弹（第 6 轮 0.4160，第 7 轮 0.4208），训练 loss 与验证 loss 的差距扩大，呈现典型过拟合特征。因此，采用 early stopping 能在性能峰值处保存模型，避免继续训练导致泛化能力下降。

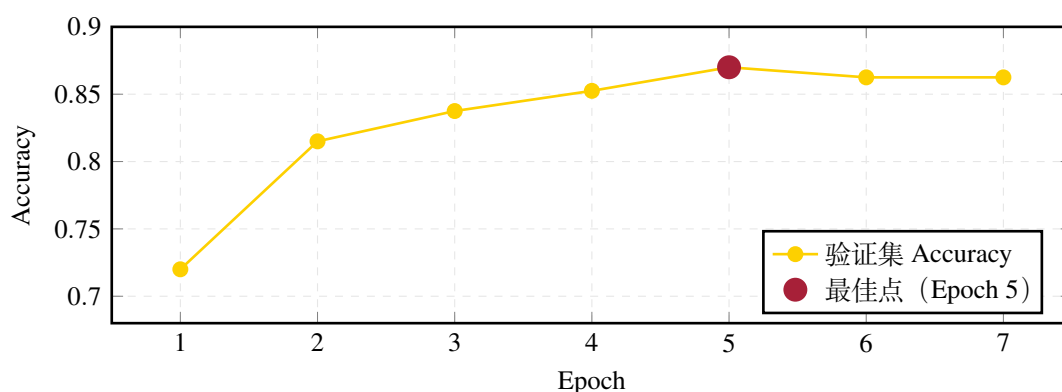


图 3: 验证集 Accuracy 随训练轮次变化曲线

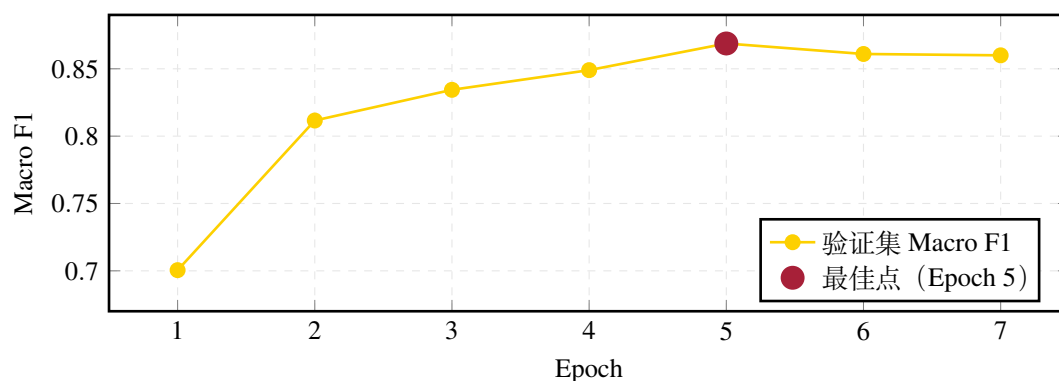


图 4: 验证集 Macro F1 随训练轮次变化曲线

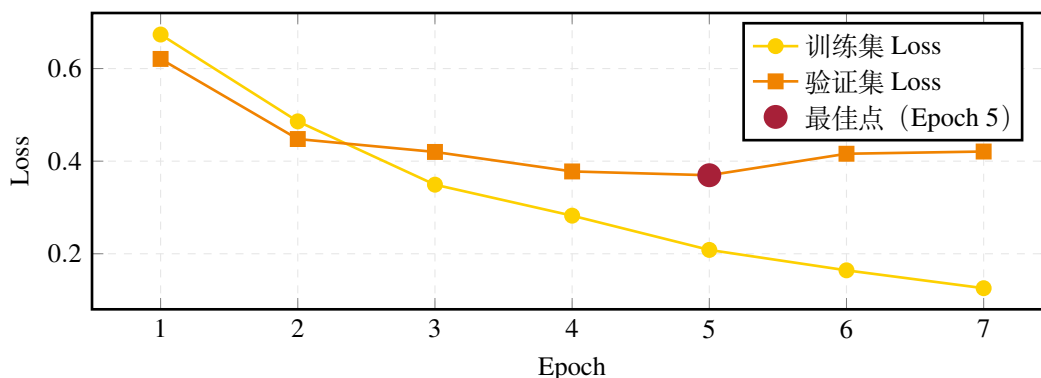


图 5: 训练集与验证集 Loss 随训练轮次变化曲线

最佳模型在验证集上的混淆矩阵如表 1 所示。模型对非谣言类 (label 0) 的精确率为 0.9061、召回率为 0.8578、F1 为 0.8813；对谣言类 (label 1) 的精确率为 0.8289、召回率为 0.8857、F1 为 0.8564。结果表明，模型对两类样本均有较稳定的识别能力，其中谣言类召回率较高，说明其更倾向于检出潜在谣言，符合谣言检测任务中降低漏检的需求。误判方面，32 例非谣言被误判为谣言，20 例谣言被误判为非谣言。

		预测标签	
		0 (非谣言)	1 (谣言)
真实标签	0 (非谣言)	193	32
	1 (谣言)	20	155

表 1: 最佳模型 (Epoch 5) 验证集混淆矩阵

综上，微调后的 BERTweet 模型在较小规模数据集上取得了较为鲁棒的分类性能，accuracy 与 macro-F1 均达到 0.87 左右，早停策略有效抑制了过拟合，模型在谣言检出率和非谣言精确率之间保持了合理的平衡。

2.4 判断依据的分析

本项目的判断依据主要由大语言模型生成，结合 RAG 检索证据提供辅助参考。LLM 会从可验证性、信源可信度、逻辑连贯性、情绪化语言和文本具体性等角度生成解释。

当 ML 与 LLM 判断一致时，系统会生成统一解释；当两者判断不一致时，系统会提示分歧并分析原因。页面下方展示 RAG Retrieved Evidence，包括证据来源、标签、距离和文本内容，增加判断过程的透明度。实验中也发现，RAG 检索结果会受到数据库内容影响，因此本项目将 RAG 定位为解释增强模块，而不是最终分类依据。

3 工作总结

3.1 收获与心得

通过本项目，我们完整实践了一个 AI 应用系统的设计与实现过程，包括数据处理、预训练模型微调、模型评估、RAG 检索增强、大语言模型 API 调用和前端展示。项目加深了我们对 Transformer 文本分类模型、RAG 检索机制和可解释 AI 系统设计的理解。

3.2 问题与解决

项目过程中主要遇到三个问题。

1. 数据集规模较小, BERTweet 型训练过多轮后容易过拟合。对此, 我们采用验证集 macro-F1 作为早停指标, 并保存最佳模型。
2. RAG 数据库较大且检索相关性存在波动, 因此数据库通过 GitHub Release 提供, 系统将 RAG 作为可选解释增强模块接入, 保证主系统稳定运行。
3. LLM 解释可能与小模型分类结果不一致。对此, 我们设计了多阶段分析流程, 将小模型判断、LLM 判断和分歧分析分开展示。

3.3 分工与合作

表2总结了我們小组成员的具体分工和贡献。

成员	分工	贡献
陈忠淳	项目统筹、小模型训练与评估	GitHub 仓库管理、BERTweet 微调与评估
沈润泽	RAG 模块开发	外部数据整理、ChromaDB 向量数据库构建、RAG 独立运行测试
戴铭辰	前端与大模型调用	FastAPI/Jinja2 前端搭建与 API 调用、LLM 三阶段分析流程设计
谢紫浩	报告与文档整理	实验报告撰写、README 更新、RAG 接入和最终运行检测

表 2: 小组成员分工与贡献

4 课程建议

建议后续课程中可以提供更多关于模型部署、API 调用、RAG 接入和可解释性评估的知识讲解和应用示例, 以帮助学生更好地完成从模型训练到系统实现的完整流程。增加可获取数据集的途径等, 方便学生进一步探索。